Czech ACM Student Chapter
CTU in Prague, Dept. of Computer Science and Engineering
Charles University in Prague, MFF
VŠB TU Ostrava, FEI, Dept. of Computer Science
SUT Bratislava, FEI, Dept. of Computer Science and Engineering
MUNI Brno, Faculty of Informatics



CTU Open Contest 2005

All Friends

`a.p`, `a.c`, `a.C`

Sociologists are interested in the phenomenon of "friendship". To study this property, they analyze various groups of people. For each two persons in such a group they determine whether they are friends (it is assumed that this relation is symmetric). The sociologists are mostly interested in the sets of friends. The set S of people is the set of friends if every two persons in S are friends. However, studying the sets of friends turns out to be quite complicated, since there are too many such sets. Therefore, they concentrate just on the maximal sets of friends. A set of friends S is maximal if every person that does not belong to S is not a friend with someone in S.

Your task is to determine the number of maximal sets of friends in each group. In case this number exceeds 1 000, you just need to report this — such a group is too complicated to study.

Input Specification

The input consists of several instances, separated by single empty lines.

The first line of each instance consists of two integers $1 \leq n \leq 128$ and m — number of persons in the group and number of friendship relations. Each of m following lines consists of two integers a_i and b_i ($1 \leq a_i, b_i \leq n$). This means that persons a_i and b_i ($a_i \neq b_i$) are friends. Each such relationship is described at most once.

Output Specification

The output for each instance consists of a single line containing the number of maximal sets of friends in the described group, or string "Too many maximal sets of friends." in case this number is greater than 1 000.

Sample Input

```
5 4
1 2
3 4
2 3
4 5
```

Output for Sample Input

```
4
```



Czech ACM Student Chapter
CTU in Prague, Dept. of Computer Science and Engineering
Charles University in Prague, MFF
VŠB TU Ostrava, FEI, Dept. of Computer Science
SUT Bratislava, FEI, Dept. of Computer Science and Engineering
MUNI Brno, Faculty of Informatics



CTU Open Contest 2005

Board Game

`b.p`, `b.c`, `b.C`

Alice and Bob started to play the following game: they have an $m \times n$ chessboard, with some of the fields removed. There are two chess pieces on distinct (non-removed) fields of the board. Alice always makes the first move and then she alternates with Bob in turns. Each turn consists of moving one of the pieces by one field horizontally or vertically. Both players can move any of the pieces, regardless of the piece moved in the previous turn. The piece cannot be moved to a removed field. The player that is able to move one of the pieces to the field occupied by the other one, thus capturing it, wins.

After some time, they found the game very boring — nobody could win, and the pieces just chased each other around the board. Therefore, they introduced a new rule — no player may move a piece in such a way that a position that already appeared during the game is repeated. The position is considered to be the same if the fields occupied by the pieces are the same (the pieces cannot be distinguished), regardless of who is on turn in the particular position. Additionally, they introduced a rule that the player who cannot make a legal move loses. Now the game is always finite and one of the players will surely win. Your goal is to find a winning strategy, if one exists.

Input Specification

The input consists of several instances, separated by single empty lines.

The first line of each instance consists of two integers m and n, $1 \leq m, n \leq 8$. Each of m following lines consists of n characters and determines the initial state of the chessboard. The characters are one of the following:

- "." for an empty field of the chessboard
- "#" for a removed field of the chessboard
- "P" for the field of the chessboard where one of the pieces starts

There are always precisely two characters "P" in each instance.

Output Specification

The output for each instance consists of a single line containing either the string "Alice wins." if Alice has a winning strategy in the described position, or the string "Bob wins.", if she has no such strategy.

Sample Input

```
4 4
P.##
..##
##..
##.P

1 5
P...P
```

Output for Sample Input

```
Alice wins.
Bob wins.
```



Czech ACM Student Chapter
CTU in Prague, Dept. of Computer Science and Engineering
Charles University in Prague, MFF
VŠB TU Ostrava, FEI, Dept. of Computer Science
SUT Bratislava, FEI, Dept. of Computer Science and Engineering
MUNI Brno, Faculty of Informatics



CTU Open Contest 2005

Crane

`c.p`, `c.c`, `c.C`

ACM has bought a new crane[1]. The crane consists of n segments of various lengths, connected by flexible joints. The end of the i-th segment is joined to the beginning of the $i+1$-th one, for $1 \leq i < n$. The beginning of the first segment is fixed at point with coordinates $(0, 0)$ and its end at point with coordinates $(0, w)$, where w is the length of the first segment. All of the segments lie always in one plane, and the joints allow arbitrary rotation in that plane. After series of unpleasant accidents, it was decided that software that controls the crane must contain a piece of code that constantly checks the position of the end of crane, and stops the crane if a collision should happen.

Your task is to write a part of this software that determines the position of the end of the n-th segment after each command. The state of the crane is determined by the angles between consecutive segments. Initially, all of the angles are straight, i.e., 180°. The operator issues commands that change the angle in exactly one joint.

Input Specification

The input consists of several instances, separated by single empty lines.

The first line of each instance consists of two integers $1 \leq n \leq 10\,000$ and $c \geq 0$ separated by a single space — the number of segments of the crane and the number of commands. The second line consists of n integers $l_1, \ldots, l_n$ $(1 \leq l_i \leq 100)$ separated by single spaces. The length of the i-th segment of the crane is l_i. The following c lines specify the commands of the operator. Each line describing the command consists of two integers s and a $(1 \leq s < n, 0 \leq a \leq 359)$ separated by a single space — the order to change the angle between the s-th and the $s+1$-th segment to a degrees (the angle is measured counterclockwise from the s-th to the $s+1$-th segment).

Output Specification

The output for each instance consists of c lines. The i-th of the lines consists of two rational numbers x and y separated by a single space — the coordinates of the end of the n-th segment after the i-th command, rounded to two digits after the decimal point. The output is judged to be correct if each of the coordinates differs by at most 0.02 from the correct position of the end of the crane.

The outputs for each two consecutive instances must be separated by a single empty line.

---

[1] crane — jeřáb

Sample Input

```
2 1
10 5
1 90

3 2
5 5 5
1 270
2 90
```

Output for Sample Input

```
5.00 10.00

-10.00 5.00
-5.00 10.00
```



Czech ACM Student Chapter
CTU in Prague, Dept. of Computer Science and Engineering
Charles University in Prague, MFF
VŠB TU Ostrava, FEI, Dept. of Computer Science
SUT Bratislava, FEI, Dept. of Computer Science and Engineering
MUNI Brno, Faculty of Informatics



CTU Open Contest 2005

Divisors

`d.p`, `d.c`, `d.C`

Your task in this problem is to determine the number of divisors of $\binom{n}{k}$. Just for fun — or do you need any special reason for such a useful computation?

Input Specification

The input consists of several instances. Each instance consists of a single line containing two integers n and k ($0 \leq k \leq n \leq 431$), separated by a single space.

Output Specification

For each instance, output a line containing exactly one integer — the number of distinct divisors of $\binom{n}{k}$. For the input instances, this number does not exceed $2^{63} - 1$.

Sample Input

```
5 1
6 3
10 4
```

Output for Sample Input

```
2
6
16
```



Czech ACM Student Chapter
CTU in Prague, Dept. of Computer Science and Engineering
Charles University in Prague, MFF
VŠB TU Ostrava, FEI, Dept. of Computer Science
SUT Bratislava, FEI, Dept. of Computer Science and Engineering
MUNI Brno, Faculty of Informatics



CTU Open Contest 2005

Emag eht htiw Em Pleh

`e.p`, `e.c`, `e.C`

This problem is a reverse case of the problem *H*. You are given the output of the problem *H* and your task is to find the corresponding input.

Sample Input

```
White: Ke1,Qd1,Ra1,Rh1,Bc1,Bf1,Nb1,a2,c2,d2,f2,g2,h2,a3,e4
Black: Ke8,Qd8,Ra8,Rh8,Bc8,Ng8,Nc6,a7,b7,c7,d7,e7,f7,h7,h6
```

Output for Sample Input

```
+---+---+---+---+---+---+---+---+
|.r.|:::|.b.|:q:|.k.|:::|.n.|:r:|
+---+---+---+---+---+---+---+---+
|:p:|.p.|:p:|.p.|:p:|.p.|:::|.p.|
+---+---+---+---+---+---+---+---+
|...|:::|.n.|:::|...|:::|...|:p:|
+---+---+---+---+---+---+---+---+
|:::|...|:::|...|:::|...|:::|...|
+---+---+---+---+---+---+---+---+
|...|:::|...|:::|.P.|:::|...|:::|
+---+---+---+---+---+---+---+---+
|:P:|...|:::|...|:::|...|:::|...|
+---+---+---+---+---+---+---+---+
|.P.|:::|.P.|:P:|...|:P:|.P.|:P:|
+---+---+---+---+---+---+---+---+
|:R:|.N.|:B:|.Q.|:K:|.B.|:::|.R.|
+---+---+---+---+---+---+---+---+
```



Czech ACM Student Chapter
CTU in Prague, Dept. of Computer Science and Engineering
Charles University in Prague, MFF
VŠB TU Ostrava, FEI, Dept. of Computer Science
SUT Bratislava, FEI, Dept. of Computer Science and Engineering
MUNI Brno, Faculty of Informatics



CTU Open Contest 2005

Failing Roads

`f.p`, `f.c`, `f.C`

There are n towns in the kingdom of Failland. In the past, Failland used to have an excellent system of roads, connecting each town directly to each other, without any two roads crossing — they used a complicated system of bridges to ensure this. Recently, however, the Road-workers Union split into n independent divisions, one for each town. Due to a huge aversion between divisions, each division strictly refuses to maintain roads that lead to a town controlled by any other division. Since initially each division controlled one town only, all of the roads were soon completely broken.

The wise king of Failland has decided to improve the situation using decrees. In several decrees, he ordered some of the divisions to unite again. Whenever road-workers received such a decree, they obeyed[1] and the divisions in concern were immediately united into a single one. But since the workers are lazy and the decree did not explicitely say otherwise, the union process did not affect the roads being maintained. The united division still maintained only those roads that it used to maintain before.

Therefore, the king started to issue another type of decrees, saying that the road workers of some division must immediately repair all of the destroyed roads between the towns the division controls. He then repeated the whole process of uniting the divisions and ordering them to work several times, and finally, when only one united division remained, he considered the problem solved and went for a vacation.

However, citizens of Failland soon noticed that there is still something wrong, and that there are too few roads. After some research, they found out that whenever the workers of a division accepted an order to repair all of the destroyed roads, they also automatically supposed that they can stop maintaing the roads they maintained before, and such roads decayed quickly.

In order to persuade the king to return from the vacation and to solve the problem somehow, citizens have decided to find as many towns as possible such that no two of them are connected by a direct repaired road. They plan to send this list to the king because they feel it could help somehow. However, this task turned out to be too difficult, thus they decided to ask you for help.

[1] Well, wouldn't you obey the order when the only other option was to have your head chopped off?

Input Specification

The input consists of several scenarios. Each scenario consists of a single line. This line contains an expression that describes the sequence of king's decrees, and hence also the current state of the roads in Failland. The expression is one of the following:

- `V` stands for a single town controlled by a single division.
- `U` e_1 e_2, where e_1 and e_2 are expressions. The expressions e_1 and e_2 correspond to disjoint sets of towns, each of the sets is under control of a different division of road-workers. The expression `U` e_1 e_2 describes the situation after the king ordered these divisions to unite, i.e., the new united division now controls all the towns in e_1 and e_2, and maintains still the same set of roads (the union of the roads maintained before, as described by e_1 and e_2).
- `C` e, where e is an expression. This expression describes the situation after the division described by e received the order to take care of the roads they neglected so far. The division still controls the same set of towns, but now maintains exactly those roads they did not maintain before the order was received. Of course, only the roads connecting two towns both controlled by the same division are concerned. The roads they used to maintain before are not maintained anymore and are considered destroyed immediately.

For example, "`C U U V V V`" describes the land with three towns controlled by a single union, and with all three roads between these towns in a perfect condition, forming a triangle. The expression "`U C U U V V V C U U V V V`" describes the land with six towns formed as a union of two such triangles. The expression "`C U C U U V V V C U U V V V`" describes the same land after the decree that orders road-workers to repair the neglected roads. Now the land has six towns and 9 roads — all of the roads between all pairs of towns, except for those six that belonged to the triangles.

Each line of the input contains at most 200 000 characters.

Output Specification

The output for each scenario consists of a single line containing a single integer — the maximum number of towns such that no two of them are connected by a maintained road.

Sample Input

```
U V V
U C U V V V
C U C U U V V V C U U V V V
```

Output for Sample Input

```
2
2
3
```



Czech ACM Student Chapter
CTU in Prague, Dept. of Computer Science and Engineering
Charles University in Prague, MFF
VŠB TU Ostrava, FEI, Dept. of Computer Science
SUT Bratislava, FEI, Dept. of Computer Science and Engineering
MUNI Brno, Faculty of Informatics



CTU Open Contest 2005

Go Endgame

`g.p`, `g.c`, `g.C`

Go is an oriental board game. The goal in this game is to surround as much territory as possible by stones of your color. This game is very difficult to play for computers — the best available programs are beaten easily by a human with just a few months of practice. Some parts of the game however can be managed easily under some assumptions. This task is about the endgame stage of the go match, when boarders of all the territories are almost completely finished and the players try to squeeze last few points. Even this stage of the game is very difficult, thus we only concentrate on much simplified model. As usual, the game is played by Alice and Bob, and the alternate on the move.

We assume that Alice has a points of territory and Bob has b points. There are n separated regions such that moves in each of the regions do not affect the moves in other regions. Suppose it is Alice's turn. The endgame proceeds as follows: Alice chooses a region and plays there. Bob responds to this move in the same region, then Alice responds to the Bob's move, and so on, until a settled state is reached. The player who is on turn then chooses another region and plays there, and the game proceeds in the same way until all regions are settled.

The player who starts to play in a region has an advantage and usually gains more points there than the other player. To model this, for the i-th region we know that if Alice starts playing there, she gains a_i points and Bob gains nothing, while if Bob starts playing there, he gains b_i points and Alice gains nothing.

Additionally, playing in this region may be *sente* for Alice or Bob. If region is sente for the player who starts playing there, he will be on turn after the region is settled, and thus he is the one to choose the next region to play in. If the region is not sente for the player who starts playing there, the other player is on turn when the region is settled. The region may be sente for both players, for only one of them, or for nobody.

Your task is, given the description of the regions, to determine the final score of the game, assuming that both players use the optimal strategy. Each player tries to have the score greater than the score of the other player by as much as possible (or smaller by as little as possible), and among the results with the same difference of scores, they prefer the one where they have the maximal score.

Input Specification

The input consists of several instances, separated by single empty lines.

The first line of each instance consists of three integers a, b ($0 \leq a, b$ and $a + b \leq 361$) and n ($0 \leq n \leq 361$) separated by single spaces — the current numbers of points of Alice and Bob, and the number of unresolved regions. Each of the following n lines describes a single region. The line describing the i-th region consists of two integers a_i and b_i ($0 \leq a_i, b_i$ and $1 \leq a_i + b_i \leq 361$), and two characters s_i and t_i, separated by single spaces. The integers a_i and b_i are the numbers of points gained by Alice and Bob by starting to play in each region. The character s_i is "S" if the region is sente for Alice, and "G" otherwise. Similarly, t_i is "S" or "G", depending on whether the region is sente for Bob or not.

Output Specification

The output for each instance consists of a single line containing two integers A and B separated by a single space. These integers are the final number of points of Alice and Bob, assuming that they both play out the rest of the game using the optimal strategy, and Alice is on turn in the beginning. For all input instances, $A + B \leq 361$.

Sample Input

```
0 0 1
5 6 G S

10 9 3
2 10 G G
1 1 S G
8 6 G S
```

Output for Sample Input

```
5 0
19 19
```



Czech ACM Student Chapter
CTU in Prague, Dept. of Computer Science and Engineering
Charles University in Prague, MFF
VŠB TU Ostrava, FEI, Dept. of Computer Science
SUT Bratislava, FEI, Dept. of Computer Science and Engineering
MUNI Brno, Faculty of Informatics



CTU Open Contest 2005

Help Me with the Game

`h.p`, `h.c`, `h.C`

Your task is to read a picture of a chessboard position and print it in the chess notation.

Input Specification

The input consists of an ASCII-art picture of a chessboard with chess pieces on positions described by the input. The pieces of the white player are shown in upper-case letters, while the black player's pieces are lower-case letters. The letters are one of "`K`" (King), "`Q`" (Queen), "`R`" (Rook), "`B`" (Bishop), "`N`" (Knight), or "`P`" (Pawn). The chessboard outline is made of plus ("`+`"), minus ("`-`"), and pipe ("`|`") characters. The black fields are filled with colons ("`:`"), white fields with dots ("`.`").

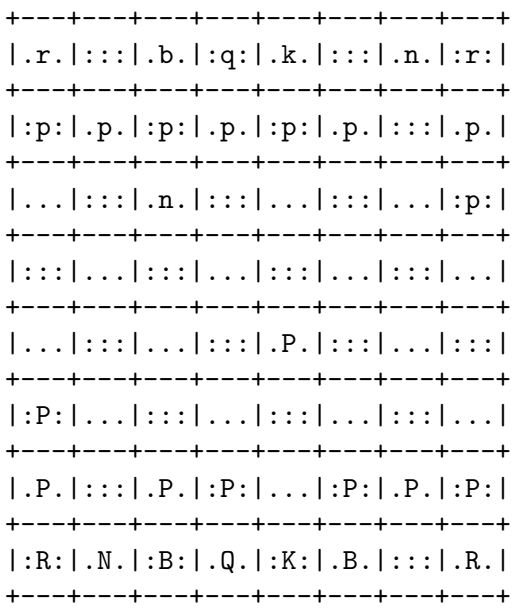
Output Specification

The output consists of two lines. The first line consists of the string "`White: `", followed by the description of positions of the pieces of the white player. The second line consists of the string "`Black: `", followed by the description of positions of the pieces of the black player.

The description of the position of the pieces is a comma-separated list of terms describing the pieces of the appropriate player. The description of a piece consists of a single upper-case letter that denotes the type of the piece (except for pawns, for that this identifier is omitted). This letter is immediatelly followed by the position of the piece in the standard chess notation — a lower-case letter between "`a`" and "`h`" that determines the column ("`a`" is the leftmost column in the input) and a single digit between 1 and 8 that determines the row (8 is the first row in the input).

The pieces in the description must appear in the following order: King("`K`"), Queens ("`Q`"), Rooks ("`R`"), Bishops ("`B`"), Knights ("`N`"), and pawns. Note that the numbers of pieces may differ from the initial position because of capturing the pieces and the promotions of pawns. In case two pieces of the same type appear in the input, the piece with the smaller row number must be described before the other one if the pieces are white, and the one with the larger row number must be described first if the pieces are black. If two pieces of the same type appear in the same row, the one with the smaller column letter must appear first.

Sample Input



Output for Sample Input

```
White: Ke1,Qd1,Ra1,Rh1,Bc1,Bf1,Nb1,a2,c2,d2,f2,g2,h2,a3,e4
Black: Ke8,Qd8,Ra8,Rh8,Bc8,Ng8,Nc6,a7,b7,c7,d7,e7,f7,h7,h6
```



Czech ACM Student Chapter
CTU in Prague, Dept. of Computer Science and Engineering
Charles University in Prague, MFF
VŠB TU Ostrava, FEI, Dept. of Computer Science
SUT Bratislava, FEI, Dept. of Computer Science and Engineering
MUNI Brno, Faculty of Informatics



CTU Open Contest 2005

IVXLCDM

`i.p`, `i.c`, `i.C`

Roman numerals are represented by a string consisting of characters "I", "V", "X", "L", "C", "D" and "M". These symbols represent the decimal values 1, 5, 10, 50, 100, 500 and 1000, respectively. To represent other values, these symbols, and multiples where necessary, are concatenated, with the smaller-valued symbols written further to the right. For example, the number 3 is represented as "III", and the value 73 is represented as "LXXIII". The exceptions to this rule occur for numbers having units values of 4 or 9, tens values of 40 or 90, and hundreds values of 400 and 900. For these cases, the Roman numeral representations are "IV" (4), "IX" (9), "XL" (40), "XC" (90), "CD" (400) and "CM" (900). So the Roman numeral representations for 24, 39, 44, 49, and 94 are "XXIV", "XXXIX", "XLIV", "XLIX", and "XCIV", respectively. Note that at most three consecutive characters "I", "X" or "C" may appear in the numeral, and each numeral contains characters "V", "L" and "D" at most once, but arbitrarily many "M"'s may appear in it. Also, it is forbidden to use, e.g., "IC" for 99.

In middle-ages, the year a building was constructed was often indicated in an inscription: some of the characters in the inscription were emphasized, and when these characters were concatenated, they formed the number of the year in roman numerals. For example, a building with inscription "Matfyz is the best schooL In prague" was clearly founded in year "MLI", i.e., 1051. However, the inscription often gets damaged and it is no longer clear which of the letters were emphasized. The archaeologists would in that case like to know at least the latest date in that the building could be constructed, by determining the largest numeral that can be encoded in an inscription in this way. For example, if they find an inscription "matfyz is the best school in prague", they know that the building was founded at latest in year "MCLI", i.e., 1151.

Input Specification

The input consists of several nonempty lines $l_1, \ldots, l_n$. Each of the lines consists of lowercase letters and spaces. The length of each line is at most 10 000 characters.

Output Specification

For each line l_i of the input, write a line consisting of a single integer to the output. This integer must be the largest roman numeral that can be encoded in the inscription given by l_i, or 0 if no roman numeral can be encoded in it.

Sample Input	Output for Sample Input
`matfyz is the best school in prague`	`1151`
`no year`	`0`